

JOBSHEET PRAKTIKUM

Mata Kuliah: Pemrograman Berbasis Framework

Pertemuan: Server Side Rendering (SSR)

Framework: Next.js

A. Capaian Pembelajaran (CPMK)

Mahasiswa mampu:

1. Menjelaskan konsep Server Side Rendering (SSR).
 2. Membedakan SSR dengan Client Side Rendering (CSR).
 3. Mengimplementasikan `getServerSideProps`.
 4. Mengelola data melalui props pada SSR.
 5. Menganalisis perbedaan performa SSR vs CSR melalui DevTools.
-

B. Dasar Teori

Konsep Server Side Rendering (SSR)

Server Side Rendering adalah proses rendering HTML yang dilakukan di server sebelum dikirim ke browser.

Alur SSR:

User Request

↓

Server fetch data

↓

Server generate HTML lengkap

↓

HTML dikirim ke browser

↓

React melakukan hydration

Karakteristik SSR:

- HTML sudah lengkap saat diterima client
- Tidak ada skeleton awal

- Cocok untuk SEO
- Data diambil setiap request

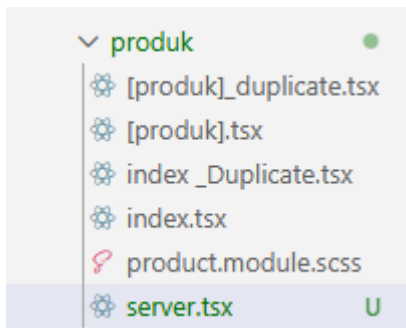
2 Perbedaan CSR vs SSR

Aspek	CSR	SSR
Rendering	Client	Server
Data Fetching	useEffect	getServerSideProps
Skeleton	Perlu	Tidak perlu
SEO	Kurang optimal	Lebih optimal
Network tab	Terlihat request API	Tidak terlihat

C. Langkah Praktikum

♦ Bagian 1 – Setup Halaman SSR

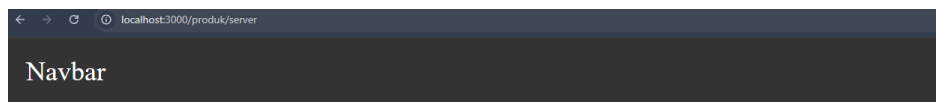
1. Buat file baru pada pages/products/server.tsx



2. Modifikasi file server.tsx :

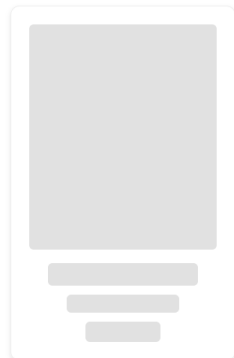
```
8.praktikum-Server_side rendering > my-app > src > pages > produk > server.tsx > default
1  import TampilanProduk from "../views/product";
2
3  Windsurf: Refactor | Explain | Generate JSDoc | X
4  const halamanProdukServer = () => {
5      return (
6          <div>
7              <h1>Halaman Produk Server</h1>
8              <TampilanProduk products={[]} />
9          </div>
10     );
11 };
12 export default halamanProdukServer;
```

3. Jalankan browser : <http://localhost:3000/produk/server>



Halaman Produk Server

Daftar Produk



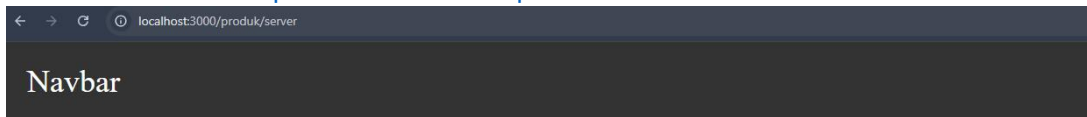
◆ Bagian 2 – Implementasi getServerSideProps pada server.tsx

8.praktikum-Server_side rendering > my-app > src > pages > produk > server.tsx > getServerSideProps

```
1  import TampilanProduk from "../views/product";
2
3  Windsurf: Refactor | Explain
4  type ProductType = {
5    id: string;
6    name: string;
7    price: number;
8    image: string;
9    category: string;
10 };
11
12 Windsurf: Refactor | Explain | Generate JSDoc | X
13 const halamanProdukServer = (props: {products: ProductType[]}) => {
14   const { products } = props;
15   return (
16     <div>
17       <h1>Halaman Produk Server</h1>
18       <TampilanProduk products={products} />
19     </div>
20   );
21 };
22 export default halamanProdukServer;
23
24 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
25 Windsurf: Refactor | Explain | X
26 export async function getServerSideProps() {
27   const res = await fetch("http://localhost:3000/api/produk");
28   const response = await res.json();
29   // console.log("Data produk yang diambil dari API:", response);
30   return {
31     props: {
32       products: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
33     },
34   };
35 }
```

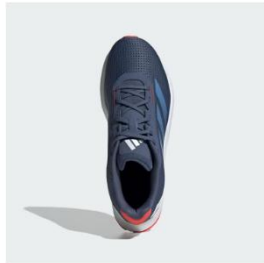
}

- Jalankan browser <http://localhost:3000/produk/server>



Halaman Produk Server

Daftar Produk



Sepatu Duramo SL

Men's Shoes

Rp 900.000,00



SEPATU SAMBA OG

Men's Shoes

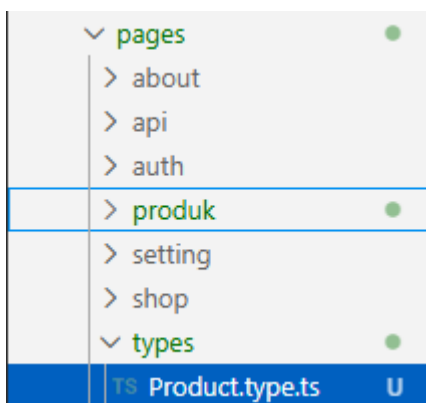
Rp 2.000.000,00

✦ Catatan:

- Jika dijalankan maka skeleton tidak muncul yang akan muncul langsung pada produknya
- Harus menggunakan full URL
- Dipanggil setiap request halaman

◆ Bagian 3 – Refactor Type (produk type)

1. Buat folder types pada folder pages dan buat file Product.type.ts



2. Modifikasi Product.type.ts

8.praktikum-Server_side rendering > my-app > src > pages > types > TS Product.type.ts > [🔗] ProductType

Windsurf: Refactor | Explain

```
1 export type ProductType = {
2   id: string;
3   name: string;
4   price: number;
5   image: string;
6   category: string;
7 };
```

3. Setelah membuat file Product.type.ts maka modifikasi pada file server.tsx menjadi

8.praktikum-Server_side rendering > my-app > src > pages > produk > server.tsx > [🔗] getServerSideProps

```
1 import TampilanProduk from "../views/product";
2 import { ProductType } from "../types/Product.type";
3
4
5
```

Windsurf: Refactor | Explain | Generate JSDoc | X

```
6 const halamanProdukServer = (props:{products:ProductType[]}) => {
7   const { products } = props;
8   return (
9     <div>
10      <h1>Halaman Produk Server</h1>
11      <TampilanProduk products={products} />
12    </div>
13  );
14 };
15
16 export default halamanProdukServer;
17
18 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
19 Windsurf: Refactor | Explain | X
20 export async function getServerSideProps() {
21   const res = await fetch("http://localhost:3000/api/produk");
22   const response = await res.json();
23   // console.log("Data produk yang diambil dari API:", response);
24   return {
25     props: {
26       products: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
27     },
28   };
29 }
```

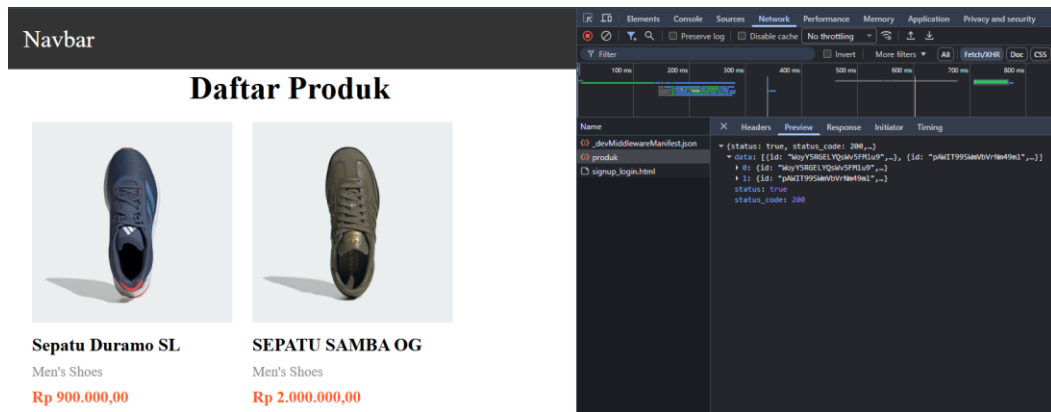
◆ Bagian 4 – Uji Perbedaan SSR vs CSR

Uji 1 – Skeleton

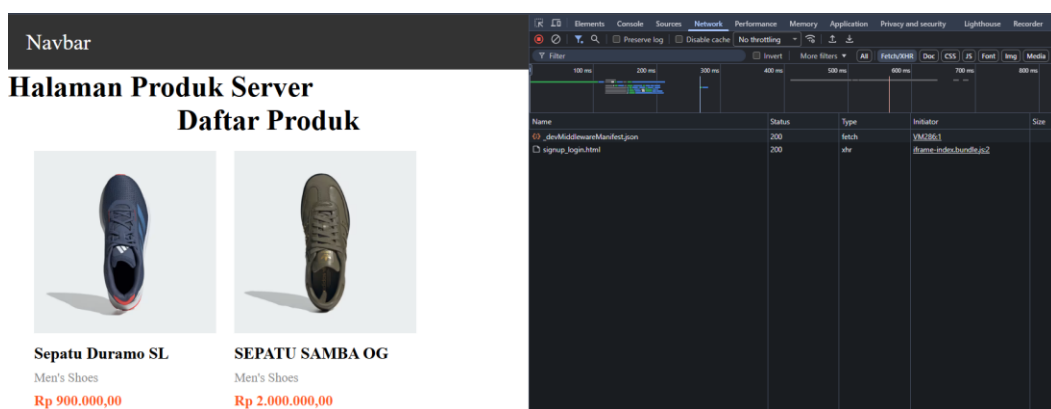
- Buka halaman CSR
- Refresh
- Skeleton muncul
- Buka halaman SSR
- Refresh
- Skeleton tidak muncul

Uji 2 – Network Tab

1. Buka DevTools → Network → XHR
2. Refresh halaman CSR
→ Request API terlihat



3. Refresh halaman SSR
→ Request API tidak terlihat



Uji 3 – Response HTML

- CSR: HTML awal kosong (berisi skeleton)
- SSR: HTML sudah berisi data produk lengkap

D. Tugas Praktikum

Tugas Individu

1. Buat 2 halaman:
 - /products (CSR)
 - /products/server (SSR)
2. Dokumentasikan:
 - Screenshot CSR
 - Screenshot SSR
 - Perbedaan Network tab

- Perbedaan View Source
3. Buat laporan analisis minimal 2 halaman.
-

E. Studi Analisis

Jawab pertanyaan berikut:

1. Mengapa SSR lebih baik untuk SEO?
2. Kapan sebaiknya menggunakan SSR?
3. Apa kekurangan SSR dibanding CSR?
4. Mengapa skeleton tidak muncul pada SSR?